

Fatih Sultan Mehmet Vakıf Üniversitesi

Computer Network Concepts

PROJECT REPORT

Backgammon Multiplayer Game

Project No: 2

Prepared by:

Saed O. S. Radi

Student ID: 2321051365

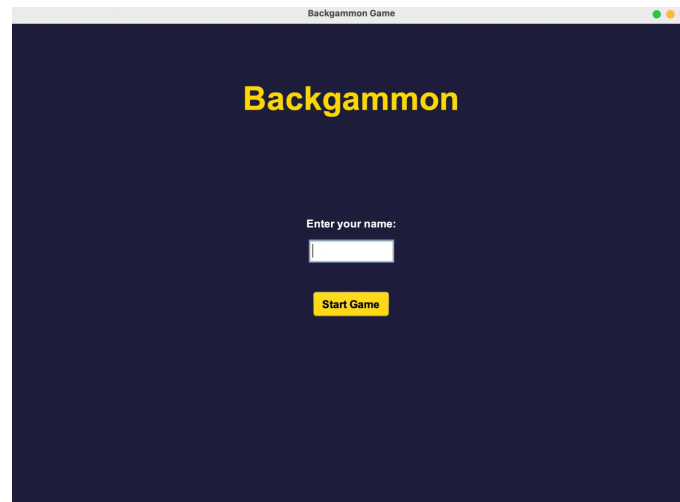
2026

Instructor: Dr. Samet Kaya
Research Assistant: Ertuğrul İslamoğlu

1. Introduction

This report documents the design and implementation of a multiplayer Backgammon game developed as part of the Computer Network Concepts course project. The project demonstrates the practical application of client-server socket programming using Java.

The game allows two players to connect over a network and play Backgammon in real-time. The server is hosted on Amazon Web Services (AWS) and manages communication between the two clients.



Start Screen of the Application

2. System Architecture

The system follows a Client-Server architecture consisting of three main components:

2.1 GameServer

The server acts as a relay between the two players. It listens on port 6000 for incoming connections. Once two players connect, it forwards each player's moves to the other player in real-time using threads.

- Hosted on AWS EC2 instance (IP: 56.228.22.133)
- Runs on port 6000
- Uses two threads — one per player — to handle simultaneous communication
- Assigns player numbers (Player 1 / Player 2) upon connection

2.2 GameClient

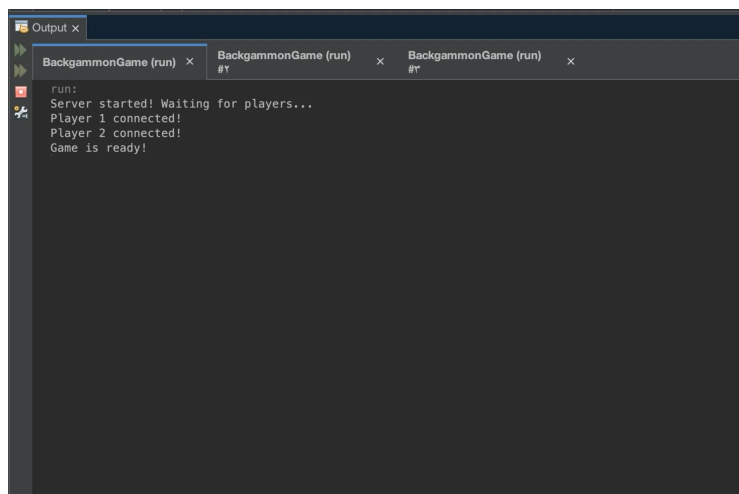
Each player runs the client application on their machine. The client connects to the server using the server's IP address and port 6000. It sends moves to the server and receives the opponent's moves in a background thread.

2.3 Communication Protocol

Moves are transmitted as plain text strings in the following format:

MOVE:fromIndex:toIndex

For example, MOVE:12:15 means a piece was moved from point 13 to point 16.



```
run:
Server started! Waiting for players...
Player 1 connected!
Player 2 connected!
Game is ready!
```

Server Console Output — Players Connected

3. Implementation

3.1 Game Logic (BackgammonBoard)

The game logic is encapsulated in the BackgammonBoard class. The board is represented as an integer array of 24 elements:

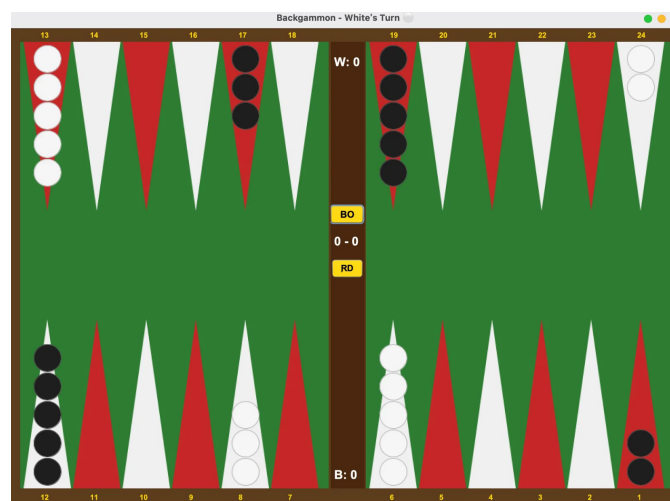
- Positive values represent Player 1 (white) pieces
- Negative values represent Player 2 (black) pieces
- Zero means the point is empty

Key methods include: isValidMove(), movePiece(), rollDice(), canBearOff(), hasWon(), and enterFromBar().

3.2 GUI Design

The graphical interface was built using Java Swing. An innovative approach was used for rendering the board: instead of drawing shapes dynamically, 48 pre-generated PNG images were created using Java's Graphics2D API — one image for each combination of triangle color (red/white), piece color (white/black), piece count (0–5), and orientation (normal/flipped).

Each of the 24 triangle points is represented as a JLabel displaying the corresponding image. When a move is made, only the image source is updated — no actual dragging occurs.

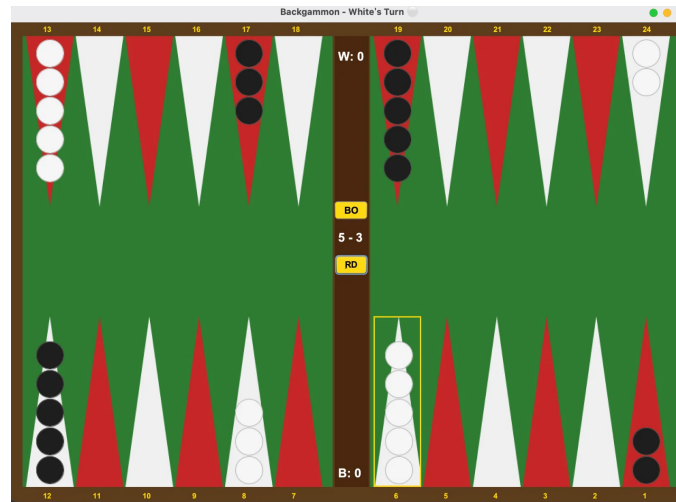


Game Board — Initial Setup with Pieces

3.3 Game Rules Implemented

- Dice roll — random values between 1 and 6
- Move validation — direction, distance, and blocking rules
- Hitting (eating) opponent's blot — piece sent to the Bar
- Bar re-entry — player must re-enter before any other move
- Bearing off — when all pieces are in the home board
- Win detection — when all 15 pieces are borne off

- Turn-based system — dice can only be rolled once per turn
- Two-dice movement — each die value can be used for a separate move



Gameplay Screenshot — Piece Being Moved

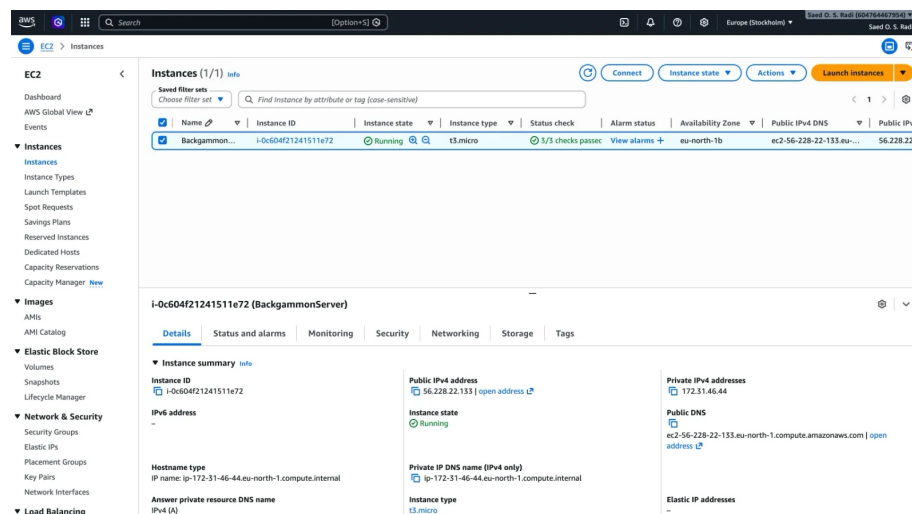
4. AWS Deployment

The GameServer was deployed on an Amazon EC2 instance running Amazon Linux 2023. The following steps were performed:

- Created an EC2 t3.micro instance (Free Tier eligible)
- Installed Java 21 (Amazon Corretto) on the instance
- Opened port 6000 in the Security Group inbound rules
- Uploaded and compiled GameServer.java via SCP
- Started the server in the background using nohup

Server IP Address: 56.228.22.133

The client applications connect to this IP address directly, satisfying the project requirement for AWS deployment.



AWS EC2 Console — Instance Running

5. Version Control (GitHub)

The project was managed using Git and hosted on a private GitHub repository. Commits were made regularly throughout the development process to track progress.

Repository: github.com/SaadOsama10/backgammon-network-project

Commits		
main	All users	All time
Commits on May 2, 2026		
Add AWS server deployment	SaadOsama10 committed 5 hours ago	b402764
Commits on Apr 17, 2026		
Fix bar entry validation and dice roll restriction	SaadOsama10 committed 2 weeks ago	ad6c1a2
Commits on Apr 16, 2026		
dd multiplayer client-server connection	SaadOsama10 committed 2 weeks ago	330f18c
Add win condition and end screen	SaadOsama10 committed 2 weeks ago	4c22e1e
Add bearing off and bar display	SaadOsama10 committed 2 weeks ago	a13d61b
Add move direction validation and bar logic	SaadOsama10 committed 2 weeks ago	d778e6b
Add dice validation, turn system, and move restrictions	SaadOsama10 committed 2 weeks ago	a8a40f7
Add dice validation, turn system, and move restrictions	SaadOsama10 committed 2 weeks ago	541d1d2
Commits on Apr 15, 2026		
Add point click logic and image update	SaadOsama10 committed 2 weeks ago	9241a96

GitHub Repository — Commits History

6. Conclusion

This project successfully demonstrates the implementation of a fully functional multiplayer Backgammon game using Java socket programming. All core requirements were met:

- Server hosted on AWS with real IP address
- Two clients communicate through the server
- Complete game rules implemented
- User-friendly graphical interface
- Start screen and end screen with replay option
- Project tracked on GitHub from the beginning

The project provided valuable hands-on experience with network programming concepts including socket communication, multi-threading, and client-server architecture.